

Figure 1: Classification of audio signals

10.1kHz, which are also 100Hz apart. This phenomenon of human auditory nerves is imitated by Mel frequency cepstral coefficients (MFCC) features. Human ears can perceive sounds that range in frequency from 20Hz to 20kHz.

Concept of framing

An audio signal is a digital signal which comprises multiple instantaneous frequencies. Thus, it is a non-stationary signal. In order to apply signal processing algorithms, we need to divide the entire signal into short duration segments, referred to as frames. These frames are typically of 20-25ms duration with an overlap of 10ms. For each frame, a stationarity assumption is made so that the feature extraction algorithms may be executed on them. A segment larger than 30ms may not preserve the stationarity principle due to the involvement of multiple frequency components, and segment length lower than 20ms may not be adequate enough to extract features properly. Researchers experiment on frame durations ranging from 20ms to 30ms, and choose the value that optimises their solution. The framing process of an audio signal is shown in Figure 2.

The number of frames for an audio signal is determined by the duration of the speech.

$$\text{Total \# frames} = \frac{\text{Total duration of the signal (in sec)}}{\text{frame size (ms)}}$$

For example, if the frame duration is 25ms and the signal is 3 seconds long, then this signal will be represented by 120 frames.

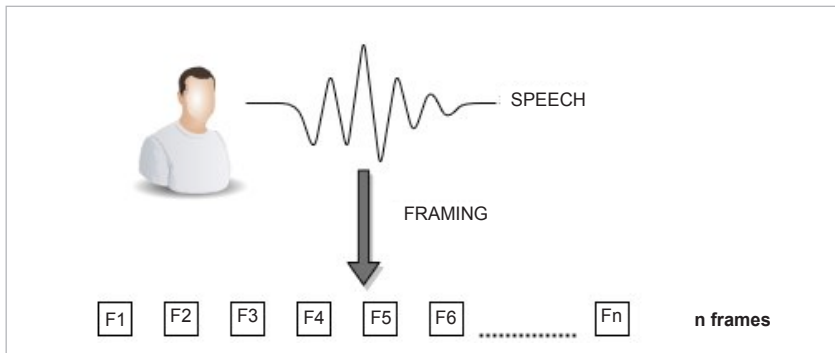


Figure 2: Illustration of framing process in audio signals [10]

Librosa

Librosa is a Python based toolkit that provides several utilities to handle audio files. It can be used to load the audio and plot the waveform of the audio file, by plotting spectrograms and varying the audio files. Let's now see how the visualisation of a sound file is done using Librosa.

To plot the waveform of an audio file, we first need to load the audio and then pass it to the plot *waveplot* function. Waveplot tells us the amplitude of sound around various time intervals. In the following code, the file name can be replaced with the actual name of the wav file.

```

import librosa
file=librosa.load('filename')
librosa.display.waveplot(file)
  
```

Amplitude envelope

The amplitude envelope extracts the maximum amplitude within each frame and merges these together. It is important to remember that the amplitude represents

the loudness of the signal. First, we split the signal into its constituent frames and find the maximum amplitude within each frame. From there, we plot the maximum amplitude in each frame along with time. The amplitude envelope can be used for onset detection or detecting the beginning of the sound. This could be external noise, someone speaking or the beginning of a musical note. The main drawback of the amplitude envelope is that it is sensitive to the outliers.

```

def amplitude_envelope(signal, frame_
length, hop_length):
    max_amplitude_frame=[]
    for i in range(0,len(signal),hop_
length):
        max_amplitude_frame.
append(max(signal[i:i+frame_length]))
    return np.array(max_amplitude_frame)
  
```

Root mean square energy (RMS)

This is somewhat similar to the amplitude envelope. As opposed to detecting the beginning of a sound, it aims at perceiving the loudness for

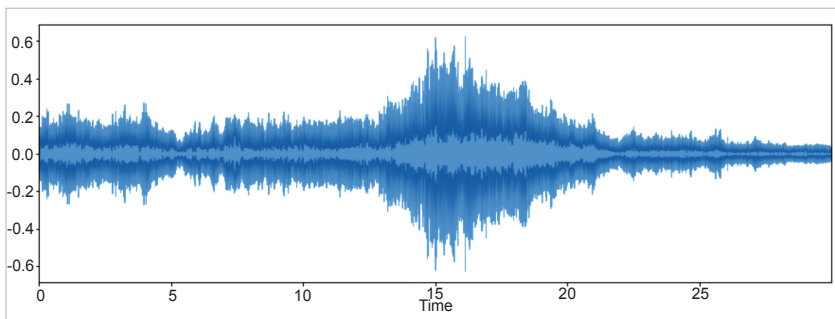


Figure 3: Time domain representation of the audio file